Support Vector Machines

Machine Learning

Nội Dung

Giới thiệu

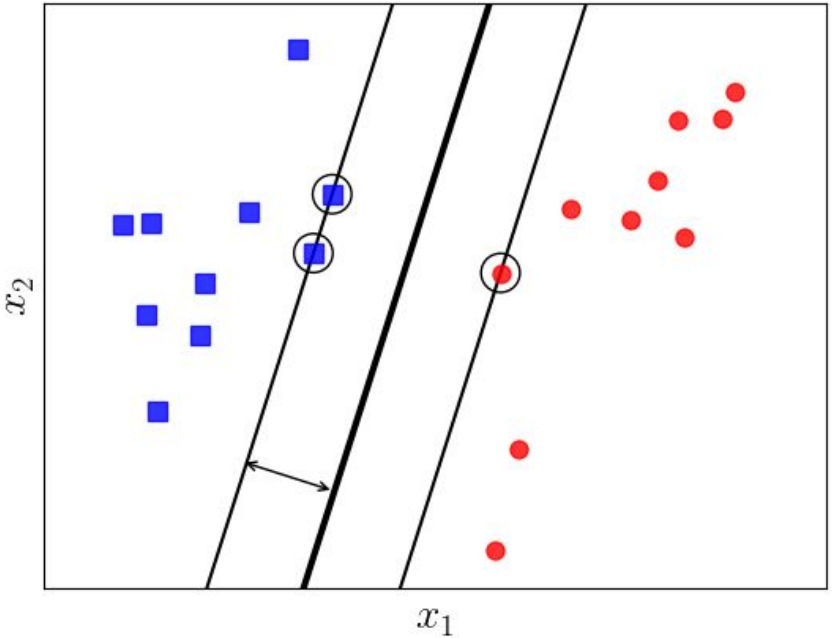
Bài toán tối ưu của SVM

Stochastic Gradient Descent

Kernel Trick

Introduction

- Support Vector Machine (SVM) là một thuật toán học có giám sát dùng cho **phân loại** và hồi quy.
- Mục tiêu của SVM là tìm ra **siêu phẳng** (hyperplane) tối ưu để phân tách dữ liệu thuộc các lớp khác nhau với một **khoảng cách lề** (margin) lớn nhất.



Source: *https://machinelearningcoban.com/*

Ideas

- Khoảng cách D từ điểm $\mathbf{x_0} = (x_1, x_2, \ldots, x_n)$ đến hyperplane $\mathrm{w} \cdot \mathrm{x} + \mathrm{b} = 0$ là:

$$D = \frac{|\mathbf{w}^T \mathbf{x_0} + b|}{\|\mathbf{w}\|_2}$$

Trong đó:

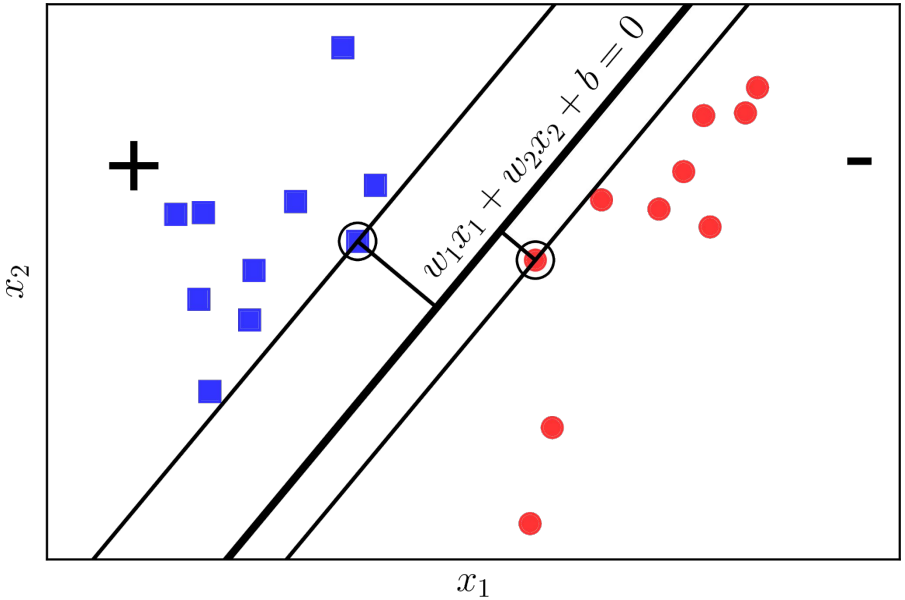
- $w^T x_0$ là tích vô hướng giữa vector pháp tuyến của hyperplane và điểm x_0.
- $\| w \|_2$ là độ dài (norm) của vector pháp tuyến w, được tính bằng công thức:

$$\|\mathbf{w}\|_2 = \sqrt{w_1^2 + w_2^2 + \cdots + w_n^2}$$

- Dấu $|\cdot|$ biểu thị giá trị tuyệt đối, giúp đảm bảo khoảng cách luôn là giá trị không âm.

Relative Positions to a Hyperplane

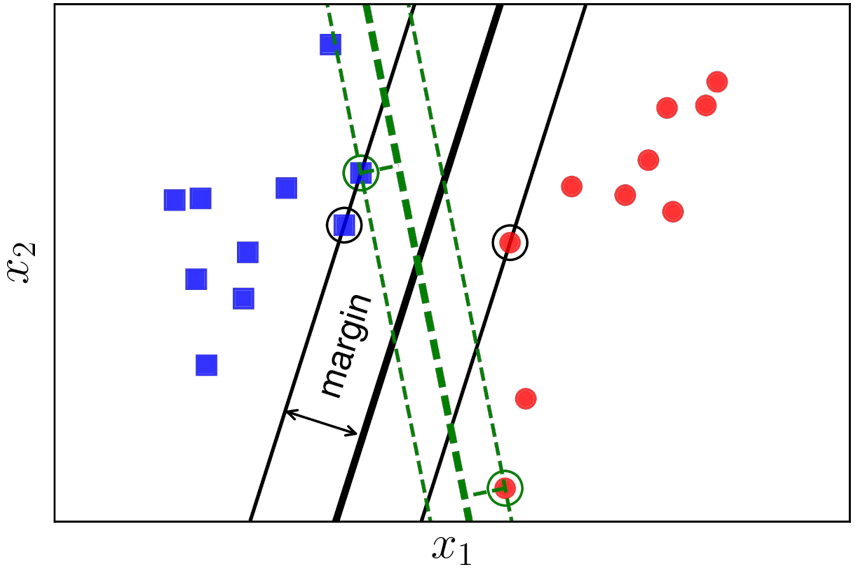
- Nếu **$w^T x+b>0$**, điểm x nằm ở **phía dương** của hyperplane.
- Nếu **$w^T x+b<0$**, điểm x nằm ở **phía âm** của hyperplane.
- Nếu **$w^T x+b=0$**, điểm x nằm **trên** hyperplane.



Source: *https://machinelearningcoban.com/*

Classification Criteria

- Tiêu chuẩn tối thiểu hóa lỗi phân loại
- Tiêu chuẩn tối đa hóa margin



Source: *https://machinelearningcoban.com/*

Formulating the Optimization Problem for SVM

- Khoảng cách từ (x_i, y_i) đến hyperplane $w^T x + b = 0$ là:

$$D_i = \frac{|\mathbf{w}^T \mathbf{x_i} + b|}{\|\mathbf{w}\|_2} = \frac{y_i(\mathbf{w}^T \mathbf{x_i} + b)}{\|\mathbf{w}\|_2}$$

- Margin:

$$margin = \min_i \frac{y_i(\mathbf{w}^T \mathbf{x_i} + b)}{\|\mathbf{w}\|_2}$$

The Optimization Problem

- Tìm $$(\mathbf{w}, b) = \arg\max_{\mathbf{w},b} \frac{1}{||\mathbf{w}||_2}$$
  $$\text{subject to:} \qquad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \forall i = 1, 2, \ldots, N$$

- Hay: $$(\mathbf{w}, b) = \arg\min_{\mathbf{w},b} \frac{1}{2}||\mathbf{w}||_2^2$$
  $$\text{subject to:} \qquad 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b) \leq 0, \forall i = 1, 2, \ldots, N$$

- Hàm mục tiêu:
  $$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \max(0, 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b))$$

Objective Function

- Hàm mục tiêu:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \max(0, 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b))$$

Trong đó:

- $\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2$: Phần này tối đa hóa margin (giảm trọng số của w).
- *C*: là tham số điều chỉnh mức độ chấp nhận lỗi phân loại.
- $\sum_{i=1}^{n} \max(0, 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b))$: là hàm **Hinge Loss**, chỉ tính lỗi phân loại khi điều kiện $y_i(w^Tx_i+b)<1$ bị vi phạm (điểm dữ liệu nằm phía sai của margin).

Hinge Loss Function

- Công thức tính Hinge Loss cho một điểm dữ liệu là:

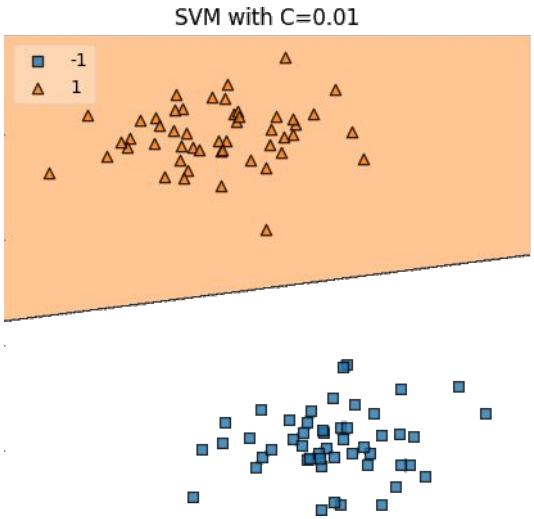
$$\text{Hinge Loss} = \max(0, 1 - y_i(w^T x_i + b))$$

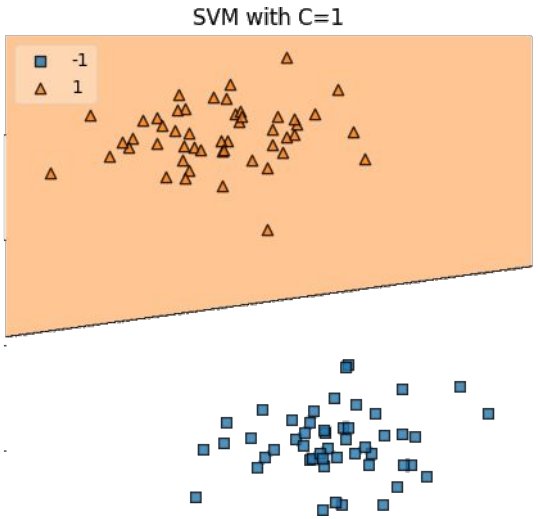
- Hinge Loss có vai trò tạo ra một **"margin rộng nhất có thể"** bằng cách phạt các điểm dữ liệu vi phạm khoảng cách tối thiểu tới siêu phẳng.
  - **Tạo margin lớn**: Hinge Loss khuyến khích SVM chọn siêu phẳng sao cho các điểm dữ liệu cách xa siêu phẳng, từ đó tạo ra một margin lớn. Margin rộng giúp tăng khả năng tổng quát của mô hình trên dữ liệu mới.
  - **Xử lý nhiễu và tránh overfitting**: Nhờ có tham số C, SVM có thể chấp nhận một số điểm vi phạm margin. Điều này giúp mô hình không bám sát quá chặt vào dữ liệu huấn luyện, đặc biệt khi có nhiễu, và từ đó tránh overfitting.

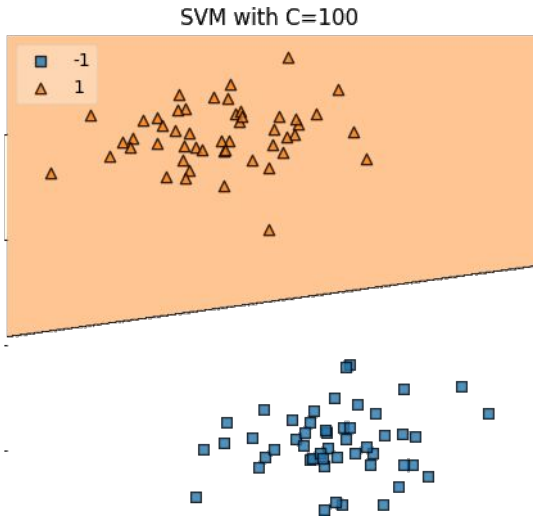
Stochastic Gradient Descent for SVM

1. *Với mỗi (x_i, y_i) tính toán Gradient cho w và b:*
   - Nếu $y_i(w^T x_i + b) \geq 1$ thì $\nabla_w = w$ và $\nabla_b = 0$
   - Nếu $y_i(w^T x_i + b) < 1$ thì $\nabla_w = w - Cy_i x_i$ và $\nabla_b = -Cy_i$
2. *Cập nhật **w** và **b** với learning rate **η**:*
   - $w \leftarrow w - \eta \nabla_w$
   - $b \leftarrow b - \eta \nabla_b$
3. *Lặp lại Bước 1 và 2 cho đến khi hội tụ*

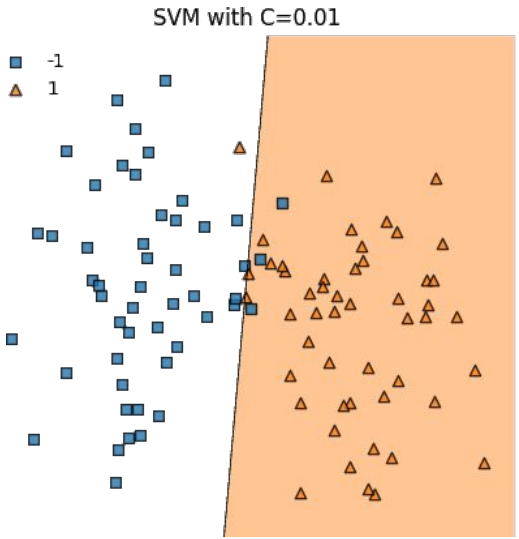
The Role of C

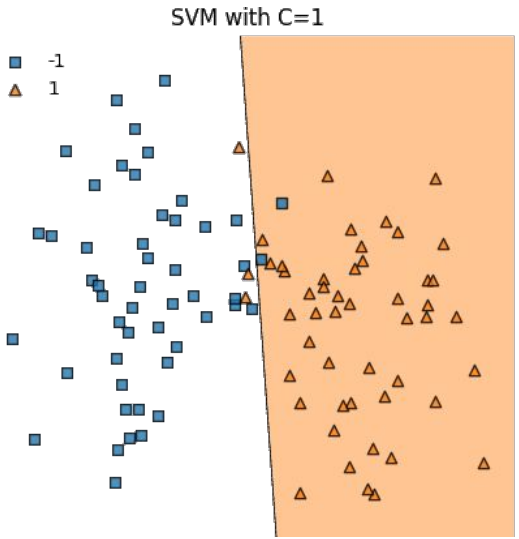


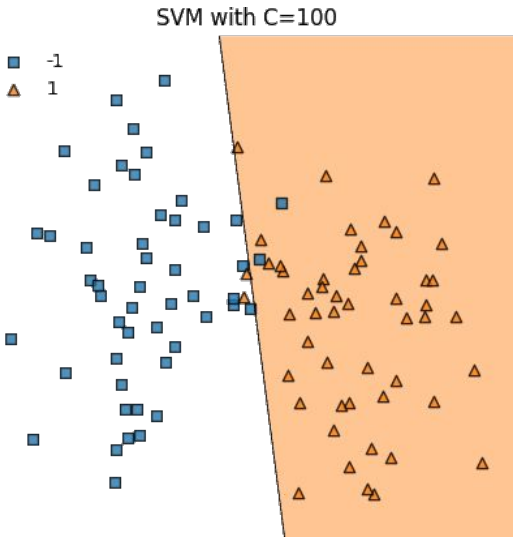




The Role of C

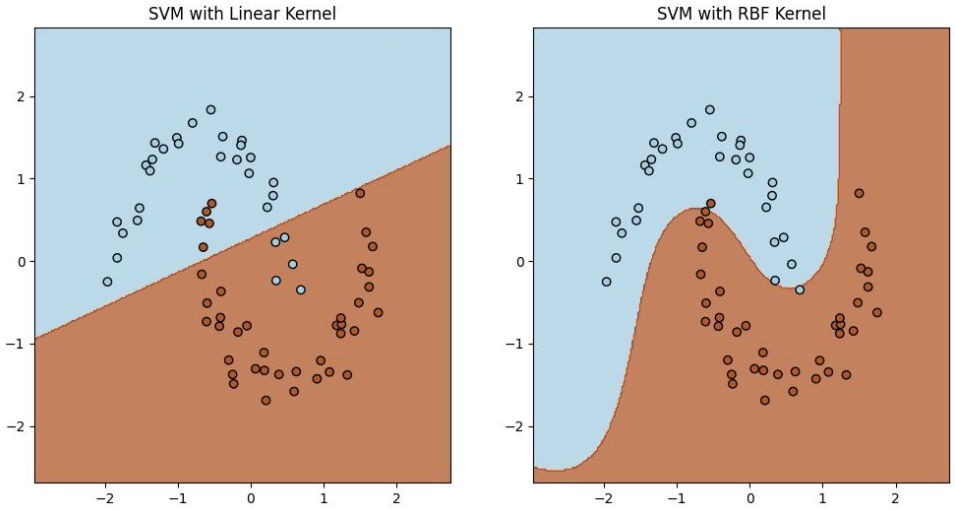






Kernel Trick

- Nếu dữ liệu không thể phân tách tuyến tính, các siêu phẳng không thể phân chia chính xác các lớp. Để giải quyết vấn đề này, SVM sử dụng **kernel trick**.
- Kernel Trick là kỹ thuật biến đổi không gian đầu vào thành một không gian đặc trưng cao hơn, trong đó các lớp có thể được phân tách tuyến tính.

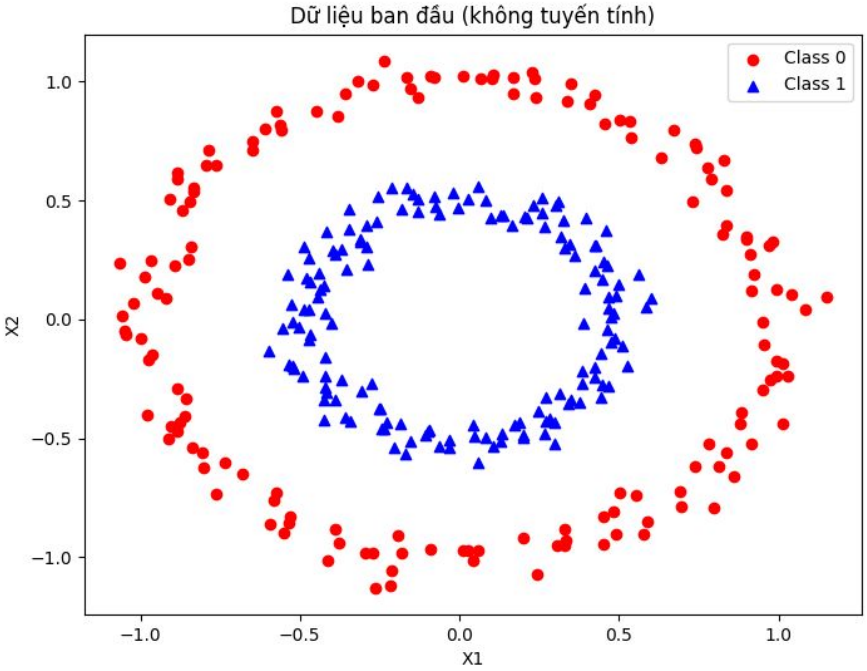


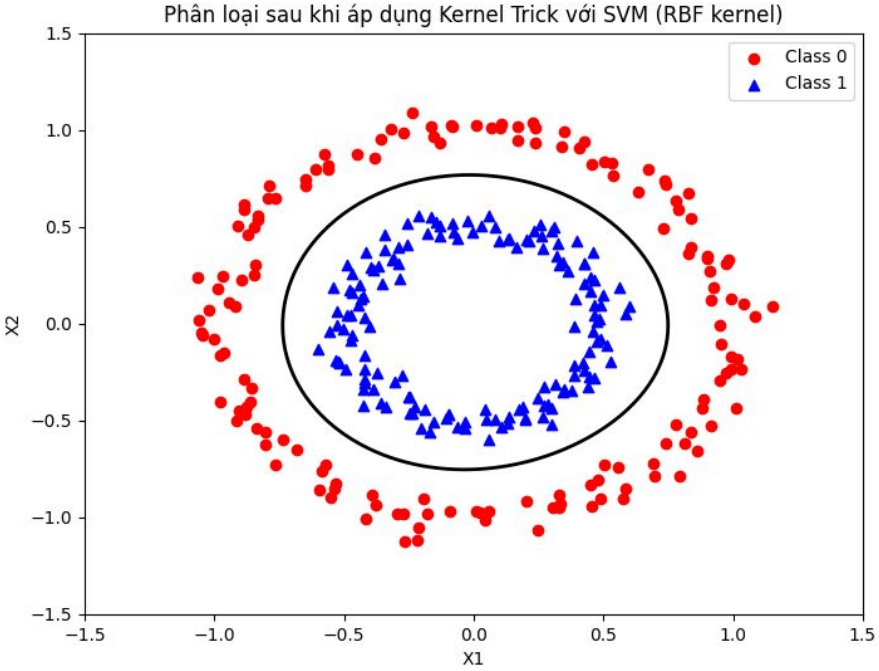
Kernels

- **Linear Kernel**:
  - Thích hợp khi dữ liệu có thể phân tách tuyến tính. Phương pháp này nhanh và hiệu quả cho các bài toán không quá phức tạp.
- **Polynomial Kernel**:
  - Mở rộng không gian dữ liệu bằng cách sử dụng hàm đa thức, giúp tạo các vùng phân tách phức tạp hơn so với tuyến tính.
- **RBF (Radial Basis Function) Kernel**:
  - Tạo vùng ảnh hưởng xung quanh từng điểm dữ liệu, hữu ích cho dữ liệu phân bố phức tạp hoặc không có dạng tuyến tính.

$$K(x, x') = \exp\left(-\gamma \|x - x'\|^2\right)$$

Ví dụ


Dữ liệu ban đầu (không tuyến tính)
Class 0
Class 1
X1
X2
1.0
0.5
0.0
−0.5
−1.0


Phân loại sau khi áp dụng Kernel Trick với SVM (RBF kernel)
Class 0
Class 1
X1
X2
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5

Finetuning

Đánh giá và điều chỉnh SVM

- **Tìm siêu phẳng tối ưu với tham số C**:
  - Tham số **C** là một siêu tham số quan trọng, quyết định mức độ cho phép lỗi trong quá trình phân loại.
  - **Giá trị lớn của C**: Khi C lớn, SVM cố gắng giảm thiểu lỗi phân loại, tạo ra biên phân chia chặt hơn nhưng có thể dẫn đến overfitting.
  - **Giá trị nhỏ của C**: Khi C nhỏ, SVM cho phép một số điểm dữ liệu nằm sai bên của siêu phẳng, giúp tạo ra biên phân chia rộng hơn, ít nhạy cảm với nhiễu, nhưng có thể underfit.
- **Lựa chọn Kernel**:
  - Việc lựa chọn kernel phụ thuộc vào đặc điểm của dữ liệu và bài toán. Thử nghiệm với nhiều loại kernel có thể giúp tìm ra loại phù hợp nhất.
  - Thông thường, RBF kernel là lựa chọn tốt cho dữ liệu phức tạp, trong khi Linear kernel thích hợp với dữ liệu có thể phân tách tuyến tính.
- **Cross-Validation**:
  - Sử dụng k-fold cross-validation để chọn ra tham số C, gamma, và kernel tối ưu.
  - Cross-validation giúp đảm bảo mô hình không bị overfit hoặc underfit, tăng cường khả năng tổng quát khi áp dụng trên dữ liệu mới.

Tài Liệu Tham Khảo

http://machinelearningcoban.com/